



República Bolivariana de Venezuela
Ministerio del Poder Popular para la Educación Universitaria
Universidad Nacional Experimental de las Telecomunicaciones e
Informática
Unidad Curricular: **Base de datos II**
Sesión Didáctica 3

Evaluación 3 NodeJs + TypeScript + Tailwind + MongoDB

Profesor: Inmaculada Maldonado

Alumnos:

C.I. V-17.205.680 Ana Contreras

C.I. V-30.730.461 Diana Sierra

C.I. V-19.064.945 Darwin Colmenares

Trayecto 3 – PNF Ingeniería en Informática

Caracas, 22 febrero de 2026

ÍNDICE

ÍNDICE	2
INTRODUCCIÓN	3
CREACIÓN DEL SISTEMA DE GESTIÓN ESCOLAR (CRUD)	4
Node.js + TypeScript + Tailwind CSS + MongoDB	4
1. REQUISITOS PREVIOS	4
2. ESTRUCTURA INICIAL Y CONFIGURACIÓN	5
3. CONFIGURACIÓN DEL ENTORNO	7
4. MODELADO DE DATOS (MONGODB)	7
5. POBLAR BASE DE DATOS (SEEDING)	8
6. DESARROLLO DEL BACKEND (API)	11
7. DESARROLLO DEL FRONTEND	13
8. CÓMO EJECUTAR EL PROYECTO	16
9. REPOSITORIO EN GITHUB	19
CONCLUSIONES	20
REFERENCIAS BIBLIOGRÁFICAS	21

INTRODUCCIÓN

En el desarrollo de aplicaciones web modernas, la combinación de entornos de ejecución como Node.js con bases de datos NoSQL como MongoDB se ha consolidado como un estándar debido a su escalabilidad y flexibilidad. La presente práctica técnica tiene como objetivo el diseño e implementación de un Sistema de Gestión Escolar basado en una arquitectura Modelo-Vista-Controlador (MVC), utilizando TypeScript para garantizar la integridad de los datos mediante el tipado estático.

A lo largo de este informe, se detalla el proceso de configuración del servidor con Express, la integración con MongoDB Atlas para el almacenamiento persistente y el desarrollo de un CRUD (Create, Read, Update, Delete) funcional. Este enfoque permite demostrar cómo la gestión de colecciones de documentos facilita el manejo de información académica, optimizando los tiempos de respuesta y simplificando la estructura de datos frente a los modelos relacionales tradicionales.

CREACIÓN DEL SISTEMA DE GESTIÓN ESCOLAR (CRUD)

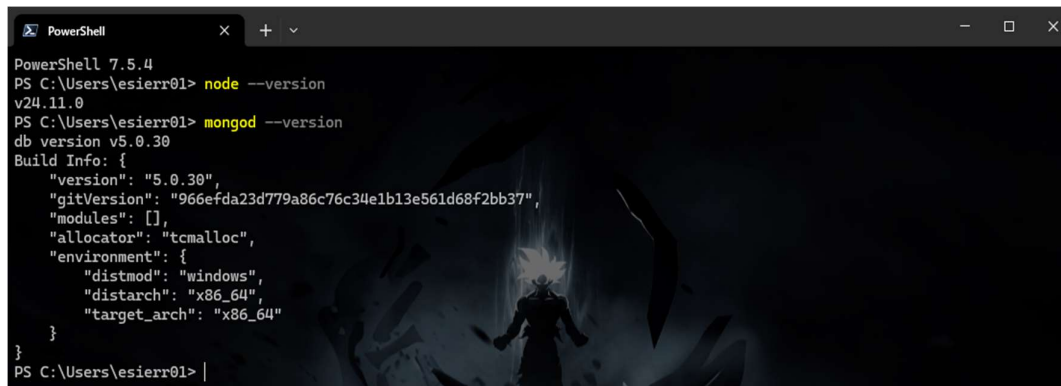
Node.js + TypeScript + Tailwind CSS + MongoDB

Aquí detallamos el proceso técnico, comandos y estructura utilizada para desarrollar la aplicación de gestión de estudiantes.

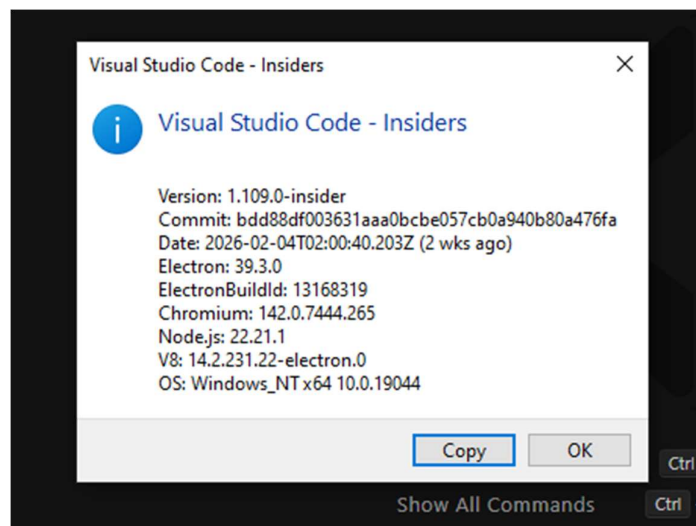
1. REQUISITOS PREVIOS

Para el desarrollo de la aplicación tenemos que contar con las siguientes herramientas instaladas en nuestro equipo:

- Node.js instalado (v16+ recomendado).
- MongoDB Community Server instalado y activo en el puerto local 27017.
- Un editor de código (VS Code recomendado).



```
PowerShell 7.5.4
PS C:\Users\esierr01> node --version
v24.11.0
PS C:\Users\esierr01> mongod --version
db version v5.0.30
Build Info: {
  "version": "5.0.30",
  "gitVersion": "966efda23d779a86c76c34e1b13e561d68f2bb37",
  "modules": [],
  "allocator": "tcmalloc",
  "environment": {
    "distmod": "windows",
    "distarch": "x86_64",
    "target_arch": "x86_64"
  }
}
```



2. ESTRUCTURA INICIAL Y CONFIGURACIÓN

- Para el desarrollo de la aplicación abrimos una terminal y creamos la carpeta para nuestro proyecto, ingresamos en ella:

```
PowerShell
PS C:\> mkdir crud-node-ts-tailwind

Directory: C:\

Mode                LastWriteTime         Length Name
----                -
d-----          22/2/2026 12:25 p. m.          crud-node-ts-tailwind

PS C:\> cd .\crud-node-ts-tailwind\
PS C:\crud-node-ts-tailwind> |
```

- Ahora ejecutamos para inicializar el proyecto: **npm init -y**

```
PowerShell
PS C:\crud-node-ts-tailwind> npm init -y
Wrote to C:\crud-node-ts-tailwind\package.json:

{
  "name": "crud-node-ts-tailwind",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

- Continuamos con la instalación de las dependencias de producción: **npm install express mongoose dotenv cors**

```
PowerShell
PS C:\crud-node-ts-tailwind> npm install express mongoose dotenv cors

added 85 packages, and audited 86 packages in 24s

25 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities
PS C:\crud-node-ts-tailwind> |
```

- Instalamos las dependencias de desarrollo (TypeScript y herramientas): **npm install -D typescript ts-node nodemon @types/node @types/express @types/cors @types/mongoose**

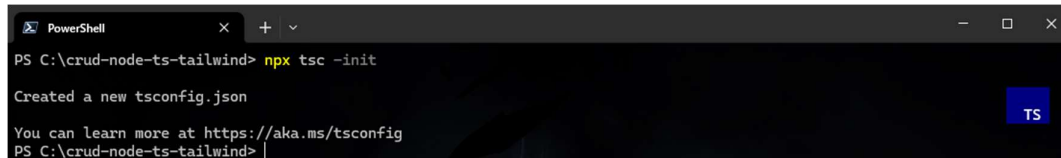
```
PowerShell
PS C:\crud-node-ts-tailwind> npm install -D typescript ts-node nodemon @types/node @types/express @types/cors @types/mongoose

added 58 packages, and audited 144 packages in 19s

30 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities
PS C:\crud-node-ts-tailwind> |
```

- Inicializamos configuración de TypeScript: `npx tsc -init`



```
PowerShell
PS C:\crud-node-ts-tailwind> npx tsc -init
Created a new tsconfig.json
You can learn more at https://aka.ms/tsconfig
PS C:\crud-node-ts-tailwind>
```

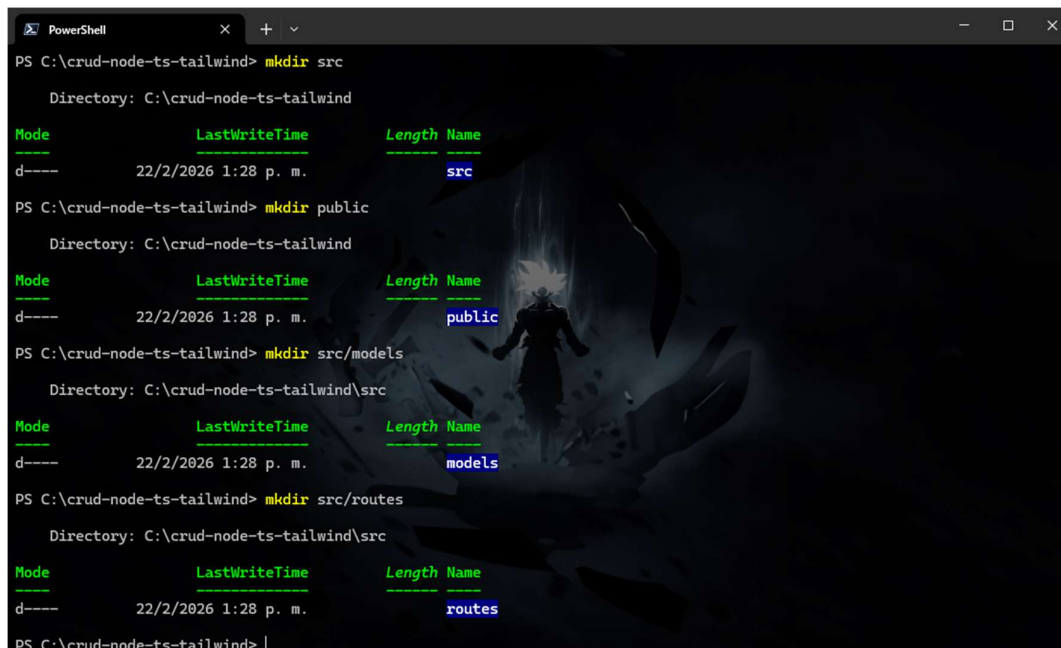
- Ahora creamos las carpetas base de nuestro proyecto ejecutando:

`mkdir src`

`mkdir public`

`mkdir src/models`

`mkdir src/routes`



```
PowerShell
PS C:\crud-node-ts-tailwind> mkdir src
Directory: C:\crud-node-ts-tailwind

Mode                LastWriteTime         Length Name
----                -
d-----         22/2/2026 1:28 p. m.             src

PS C:\crud-node-ts-tailwind> mkdir public
Directory: C:\crud-node-ts-tailwind

Mode                LastWriteTime         Length Name
----                -
d-----         22/2/2026 1:28 p. m.             public

PS C:\crud-node-ts-tailwind> mkdir src\models
Directory: C:\crud-node-ts-tailwind\src

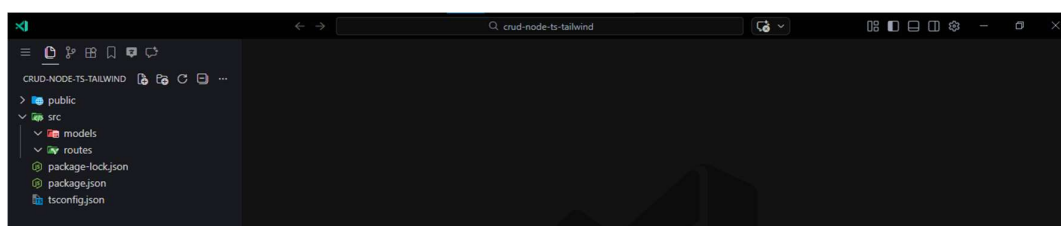
Mode                LastWriteTime         Length Name
----                -
d-----         22/2/2026 1:28 p. m.             models

PS C:\crud-node-ts-tailwind> mkdir src\routes
Directory: C:\crud-node-ts-tailwind\src

Mode                LastWriteTime         Length Name
----                -
d-----         22/2/2026 1:28 p. m.             routes

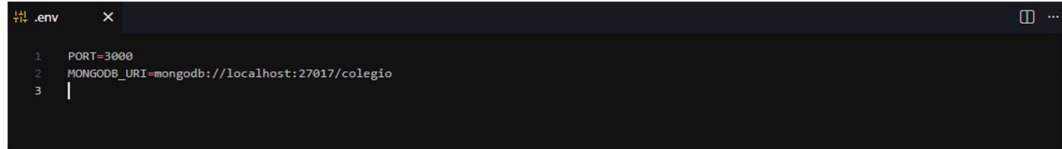
PS C:\crud-node-ts-tailwind>
```

- Abrimos el directorio de nuestro proyecto con VSCODE:



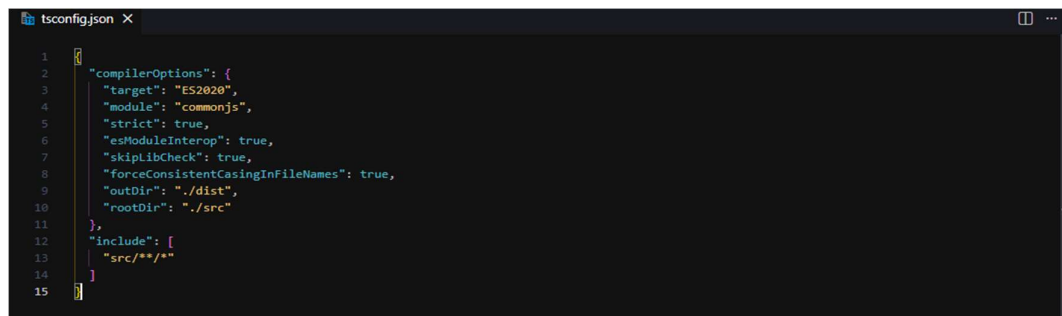
3. CONFIGURACIÓN DEL ENTORNO

- Creamos el archivo .env para almacenar variables sensibles como la URI de la base de datos.



```
.env
1 PORT=3000
2 MONGODB_URI=mongodb://localhost:27017/colegio
3
```

- Creamos el archivo: tsconfig.json para configurar el compilador de TypeScript para manejar módulos y rutas.

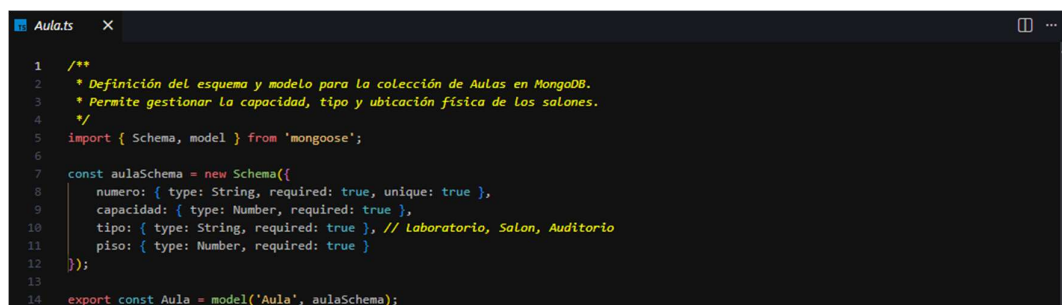


```
tsconfig.json
1
2 "compilerOptions": {
3   "target": "ES2020",
4   "module": "commonjs",
5   "strict": true,
6   "esModuleInterop": true,
7   "skipLibCheck": true,
8   "forceConsistentCasingInFileNames": true,
9   "outDir": "./dist",
10  "rootDir": "./src"
11 },
12 "include": [
13   "src/**/*"
14 ]
15
```

4. MODELADO DE DATOS (MONGODB)

Se crearon los 5 modelos solicitados utilizando Mongoose para definir el esquema de las colecciones en MongoDB.

- src/models/Estudiante.ts (CRUD Principal): Campos: nombre, cedula, carrera, trayecto_actual.
- src/models/Profesor.ts: Campos: nombre, especialidad, email, telefono.
- src/models/Materia.ts: Campos: nombre, codigo, credits, semestre.
- src/models/Aula.ts: Campos: numero, capacidad, tipo, piso.
- src/models/Representante.ts: Campos: nombre, cedula, parentesco, telefono.



```
Aula.ts
1 /**
2  * Definición del esquema y modelo para la colección de Aulas en MongoDB.
3  * Permite gestionar la capacidad, tipo y ubicación física de los salones.
4  */
5 import { Schema, model } from 'mongoose';
6
7 const aulaSchema = new Schema({
8   numero: { type: String, required: true, unique: true },
9   capacidad: { type: Number, required: true },
10  tipo: { type: String, required: true }, // Laboratorio, Salon, Auditorio
11  piso: { type: Number, required: true }
12 });
13
14 export const Aula = model('Aula', aulaSchema);
```

```

1  /**
2   * Estructura de datos para Los estudiantes.
3   * Define Los campos básicos como nombre, cédula y carrera académica.
4   */
5  import { Schema, model } from 'mongoose';
6
7  const estudianteSchema = new Schema({
8    nombre: { type: String, required: true },
9    cedula: { type: Number, required: true, unique: true },
10   carrera: { type: String, required: true },
11   trayecto_actual: { type: Number, required: true }
12 });
13
14 export const Estudiante = model('Estudiante', estudianteSchema);
15

```

```

1  /**
2   * Modelo para Las materias o asignaturas del sistema.
3   * Contiene información sobre créditos, semestre y códigos de identificación.
4   */
5  import { Schema, model } from 'mongoose';
6
7  const materiaSchema = new Schema({
8    nombre: { type: String, required: true },
9    codigo: { type: String, required: true, unique: true },
10   creditos: { type: Number, required: true },
11   semestre: { type: Number, required: true }
12 });
13
14 export const Materia = model('Materia', materiaSchema);
15

```

```

1  /**
2   * Definición para La colección de Profesores.
3   * Almacena Los datos de contacto y La especialidad de cada docente.
4   */
5  import { Schema, model } from 'mongoose';
6
7  const profesorSchema = new Schema({
8    nombre: { type: String, required: true },
9    especialidad: { type: String, required: true },
10   email: { type: String, required: true },
11   telefono: { type: String, required: true }
12 });
13
14 export const Profesor = model('Profesor', profesorSchema);
15

```

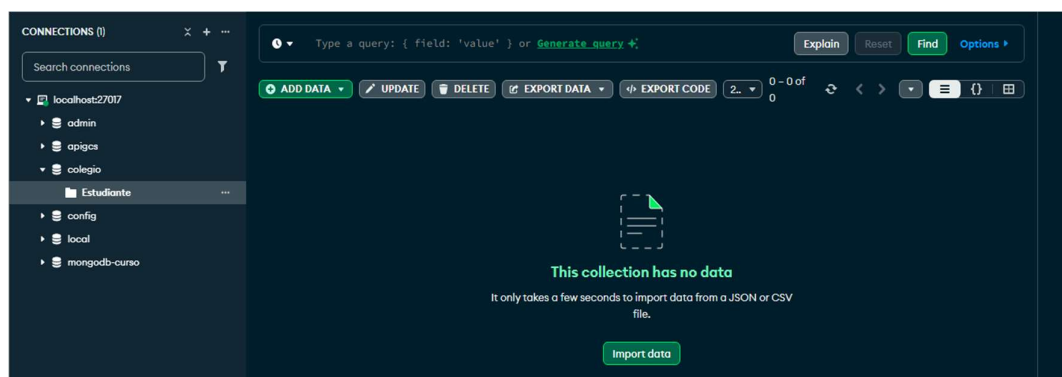
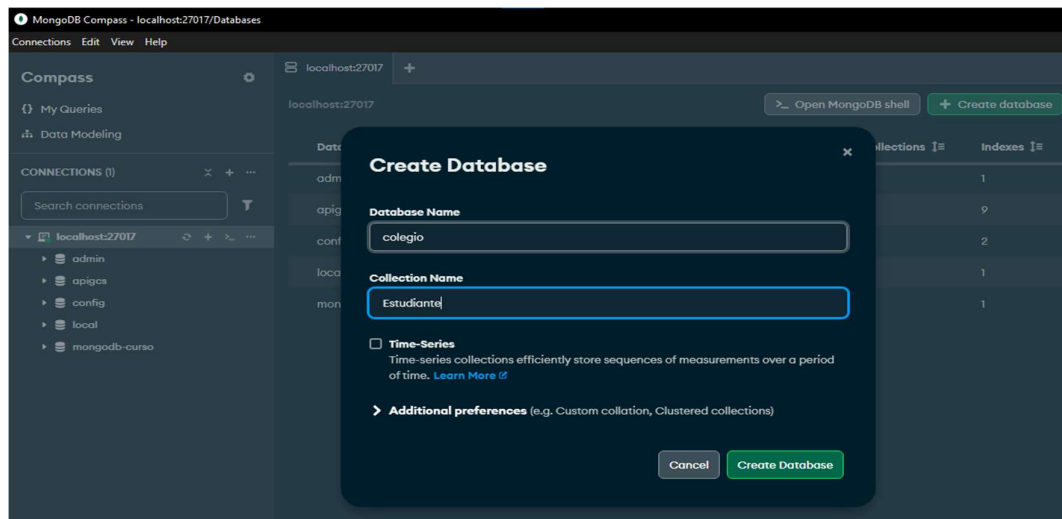
```

1  /**
2   * Esquema para Los representantes Legales o padres de Los estudiantes.
3   * Esencial para el registro de contactos de emergencia.
4   */
5  import { Schema, model } from 'mongoose';
6
7  const representanteSchema = new Schema({
8    nombre: { type: String, required: true },
9    cedula: { type: Number, required: true, unique: true },
10   parentesco: { type: String, required: true },
11   telefono: { type: String, required: true }
12 });
13
14 export const Representante = model('Representante', representanteSchema);
15

```

5. POBLAR BASE DE DATOS (SEEDING)

Primero creamos la base de datos a utilizar en nuestra aplicación “colegio” para esto utilizamos MongoDB Compass



Se creó el archivo: **src/seed.ts**, para así crear e insertar datos en las 4 colecciones (Profesores, Materias, Aulas y Representantes) que no son objetos del CRUD.

```
/**
 * Script para poblar la base de datos con información inicial, para cumplir con requerimiento
 * de la UC base de Datos II
 */
import mongoose from 'mongoose';
import dotenv from 'dotenv';
import { Profesor } from './models/Profesor';
import { Materia } from './models/Materia';
import { Aula } from './models/Aula';
import { Representante } from './models/Representante';

dotenv.config();

const MONGODB_URI = process.env.MONGODB_URI || 'mongodb://localhost:27817/colegio';

const seedData = async () => {
  try {
    await mongoose.connect(MONGODB_URI);
    console.log('Conectado a MongoDB para carga de datos...');

    // Limpiar colecciones antes de insertar
    await Profesor.deleteMany({});
    await Materia.deleteMany({});
    await Aula.deleteMany({});
    await Representante.deleteMany({});
    console.log('Colecciones limpiadas.');
```

```

// 1. Profesores
const profesores = [
  { nombre: 'Carlos Rodríguez', especialidad: 'Matemáticas', email: 'carlos.rod@escuela.edu', telefono: '0414-1234567' },
  { nombre: 'María Elena García', especialidad: 'Lengua y Literatura', email: 'maria.garcia@escuela.edu', telefono: '0412-7654321' },
  { nombre: 'José Luis Martínez', especialidad: 'Física', email: 'jose.mtz@escuela.edu', telefono: '0416-1112233' },
  { nombre: 'Ana Isabel López', especialidad: 'Química', email: 'ana.lopez@escuela.edu', telefono: '0424-4445566' }
];
await Profesor.insertMany(profesores);
console.log('Profesores insertados.');
```

```

// 2. Materias
const materias = [
  { nombre: 'Cálculo I', codigo: 'MAT101', credits: 4, semestre: 1 },
  { nombre: 'Programación Orientada a Objetos', codigo: 'PROG202', credits: 5, semestre: 2 },
  { nombre: 'Física Mecánica', codigo: 'FIS102', credits: 4, semestre: 1 },
  { nombre: 'Base de Datos I', codigo: 'BD301', credits: 3, semestre: 3 }
];
await Materia.insertMany(materias);
console.log('Materias insertadas.');
```

```

// 3. Aulas
const aulas = [
  { numero: 'L-101', capacidad: 30, tipo: 'Laboratorio', piso: 1 },
  { numero: 'S-205', capacidad: 45, tipo: 'Salon', piso: 2 },
  { numero: 'A-001', capacidad: 100, tipo: 'Auditorio', piso: 0 },
  { numero: 'S-302', capacidad: 25, tipo: 'Salon', piso: 3 }
];
await Aula.insertMany(aulas);
console.log('Aulas insertadas.');
```

```

// 4. Representantes
const representantes = [
  { nombre: 'Juan Carlos Pérez', cedula: 12345678, parentesco: 'Padre', telefono: '0414-9998877' },
  { nombre: 'Carmen Rosa Mendoza', cedula: 87654321, parentesco: 'Madre', telefono: '0412-5554433' },
  { nombre: 'Luis Alberto Gómez', cedula: 11223344, parentesco: 'Tío', telefono: '0416-6667788' },
  { nombre: 'Beatriz Adriana Silva', cedula: 55667788, parentesco: 'Madre', telefono: '0424-1119900' }
];
await Representante.insertMany(representantes);
console.log('Representantes insertados.');
```

```

console.log('¡Datos cargados exitosamente!');
process.exit(0);
} catch (error) {
  console.error('Error al cargar datos:', error);
  process.exit(1);
}
}

seedData();

```

Ahora en la terminal Ejecutamos: **`npx ts-node src/seed.ts`**

```

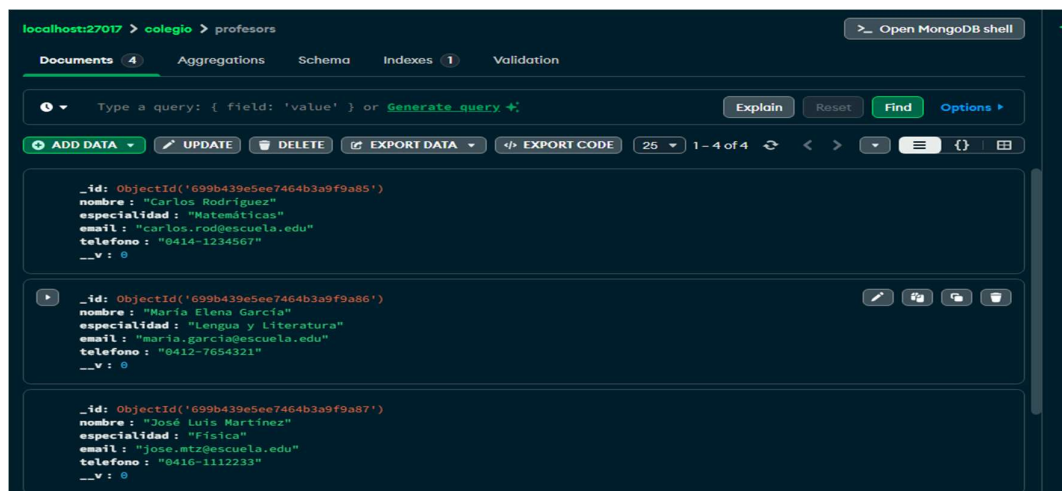
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\crud-node-ts-tailwind> npx ts-node src/seed.ts
[dotenv@17.3.1] injecting env (2) from .env -- tip: load multiple .env files with { path: ['.env.local', '.env'] }
Conectado a MongoDB para carga de datos...
Colecciones limpiadas.
Profesores insertados.
Materias insertadas.
Aulas insertadas.
Representantes insertados.
¡Datos cargados exitosamente!
PS C:\crud-node-ts-tailwind>

```

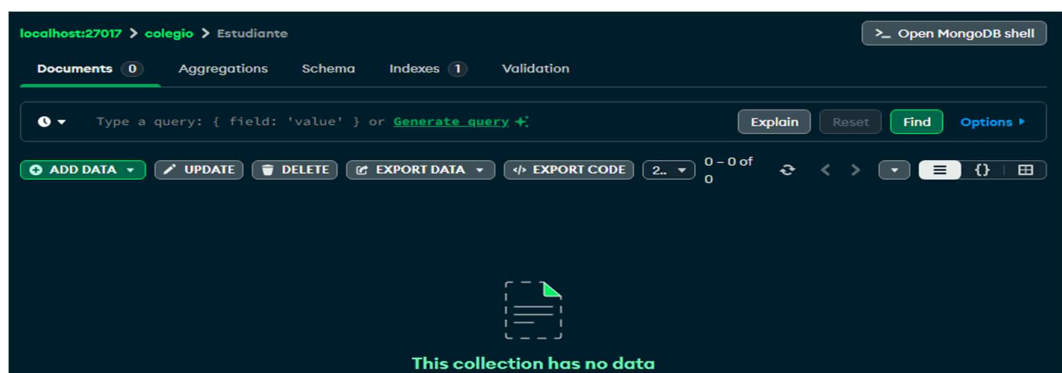
Se crearon las colecciones de Profesores, Materias, Aulas y Representantes.

Collection name	Properties	Storage size	Documents	Avg. document size	Indexes	Total index size
Estudiante	-	4.10 kB	0	0 B	1	4.10 kB
aulas	-	4.10 kB	4	92.00 B	2	8.19 kB
materias	-	4.10 kB	4	109.00 B	2	8.19 kB
profesors	-	4.10 kB	4	163.00 B	1	4.10 kB
representantes	-	4.10 kB	4	124.00 B	2	8.19 kB

En cada una de las colecciones fueron insertados los documentos con los datos del seed.



La colección Estudiante aún está vacía, ya que llenaremos con el CRUD:



6. DESARROLLO DEL BACKEND (API)

Creamos el archivo: `src/routes/estudiantes.ts`, para poder definir los endpoints GET, POST, PUT y DELETE para los estudiantes.

```
/**
 * Definición de Los puntos de entrada (endpoints) de La API para estudiantes.
 * Aquí manejamos Las operaciones CRUD: Obtener, Crear, Actualizar y Eliminar.
 */
import { Router } from 'express';
import { Estudiante } from '../models/Estudiante';

const router = Router();

// Obtener todos Los estudiantes
router.get('/', async (req, res) => {
  try {
    const estudiantes = await Estudiante.find();
    res.json(estudiantes);
  } catch (error) {
    res.status(500).json({ message: 'Error al obtener estudiantes', error });
  }
});
```

```

// Crear un estudiante
router.post('/', async (req, res) => {
  try {
    const nuevoEstudiante = new Estudiante(req.body);
    await nuevoEstudiante.save();
    res.status(201).json(nuevoEstudiante);
  } catch (error) {
    res.status(400).json({ message: 'Error al crear estudiante', error });
  }
});

// Actualizar un estudiante
router.put('/:id', async (req, res) => {
  try {
    const estudianteActualizado = await Estudiante.findByIdAndUpdate(req.params.id, req.body, { new: true });
    res.json(estudianteActualizado);
  } catch (error) {
    res.status(400).json({ message: 'Error al actualizar estudiante', error });
  }
});

// Borrar un estudiante
router.delete('/:id', async (req, res) => {
  try {
    await Estudiante.findByIdAndDelete(req.params.id);
    res.json({ message: 'Estudiante eliminado' });
  } catch (error) {
    res.status(500).json({ message: 'Error al eliminar estudiante', error });
  }
});

export default router;

```

Creamos el archivo: src/server.ts, que funciona como punto de entrada de la aplicación. Configura Express, conecta a MongoDB local y sirve los archivos estáticos de la carpeta /public.

```

/**
 * Configuración principal del servidor Express.
 * Aquí se inicializan los middlewares, la conexión a la base de datos y se definen las rutas principales.
 */
import express from 'express';
import mongoose from 'mongoose';
import dotenv from 'dotenv';
import cors from 'cors';
import path from 'path';
import estudianteRoutes from './routes/estudiantes';

dotenv.config();

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware
app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, '../public')));

// Rutas de la API
app.use('/api/estudiantes', estudianteRoutes);

// Conexión a MongoDB
mongoose.connect(process.env.MONGODB_URI || 'mongodb://localhost:27017/colegio')
  .then(() => {
    console.log('Conectado a MongoDB');
    app.listen(PORT, () => {
      console.log('Servidor corriendo en http://localhost:${PORT}');
    });
  })
  .catch((err) => {
    console.error('Error al conectar a MongoDB:', err);
  });

```

7. DESARROLLO DEL FRONTEND

Creamos el archivo: public/index.html, que es la Interfaz de usuario. Implementa con Tailwind CSS mediante CDN para un diseño moderno. Contiene el formulario y la tabla.

```
<!--
  Estructura visual de la aplicación.
  Carga Tailwind CSS para los estilos y define el contenedor para el CRUD de estudiantes.
-->
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="author" content="Ana Contreras, Diana Sierra, Darwin Colmenares">
  <title>Gestión de Estudiantes - CRUD</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap" rel="stylesheet">
  <style>
    body {
      font-family: 'Inter', sans-serif;
    }
  </style>
</head>

<body class="bg-gray-50 text-gray-900 min-h-screen">
  <div class="container mx-auto px-4 py-8">
    <header class="mb-10 text-center">
      <h1 class="text-4xl font-extrabold text-indigo-600 mb-2">Sistema Escolar (Tarea SD3)</h1>
      <p class="text-gray-600 text-sm">Gestión de Estudiantes (Node.js + TS + Tailwind + MongoDB)</p>
      <p class="text-gray-600 text-sm">UC Base de Datos II - Estudiantes: <span class="text-blue-600">Ana Contreras, Diana Sierra y Darwin Colmenares</span>
    </p>
    </header>

    <div class="grid grid-cols-1 lg:grid-cols-3 gap-8">
      <!-- Formulario -->
      <div class="lg:col-span-1">
        <div class="bg-white rounded-xl shadow-lg p-6 sticky top-8">
          <h2 id="form-title" class="text-xl font-bold mb-6 text-gray-800">Agregar Estudiante</h2>
          <form id="student-form" class="space-y-4">
            <input type="hidden" id="student-id">
            <div>
              <label class="block text-sm font-medium text-gray-700 mb-1">Nombre Completo</label>
              <input type="text" id="nombre" required class="w-full px-4 py-2 border border-gray-300 rounded-lg focus:ring-2 focus:ring-indigo-500 focus:border-indigo-500 outline-none transition-all">
            </div>
            <div>
              <label class="block text-sm font-medium text-gray-700 mb-1">Cédula</label>
              <input type="number" id="cedula" required class="w-full px-4 py-2 border border-gray-300 rounded-lg focus:ring-2 focus:ring-indigo-500 focus:border-indigo-500 outline-none transition-all">
            </div>
            <div>
              <label class="block text-sm font-medium text-gray-700 mb-1">Carrera</label>
              <input type="text" id="carrera" required class="w-full px-4 py-2 border border-gray-300 rounded-lg focus:ring-2 focus:ring-indigo-500 focus:border-indigo-500 outline-none transition-all">
            </div>
            <div>
              <label class="block text-sm font-medium text-gray-700 mb-1">Trayecto Actual</label>
              <input type="number" id="trayecto" required class="w-full px-4 py-2 border border-gray-300 rounded-lg focus:ring-2 focus:ring-indigo-500 focus:border-indigo-500 outline-none transition-all">
            </div>
            <div class="pt-2 flex gap-2">
              <button type="submit" id="submit-btn" class="flex-1 bg-indigo-600 hover:bg-indigo-700 text-white font-semibold py-2 rounded-lg transition-colors shadow-md">
                Guardar Estudiante
              </button>
              <button type="button" id="cancel-btn" class="hidden flex-1 bg-gray-200 hover:bg-gray-300 text-gray-700 font-semibold py-2 rounded-lg transition-colors">
                Cancelar
              </button>
            </div>
          </form>
        </div>
      </div>
    </div>
  </div>
</body>
```

```

<!-- Tabla de Datos -->
<div class="lg:col-span-2">
  <div class="bg-white rounded-xl shadow-lg overflow-hidden">
    <div class="p-6 border-b border-gray-100 flex justify-between items-center">
      <h2 class="text-xl font-bold text-gray-800">Lista de Estudiantes</h2>
      <span id="student-count"
        class="bg-indigo-100 text-indigo-700 px-3 py-1 rounded-full text-sm font-medium">0
        Estudiantes</span>
    </div>
    <div class="overflow-x-auto">
      <table class="w-full text-left">
        <thead class="bg-gray-50 text-gray-600 text-sm uppercase font-semibold">
          <tr>
            <th class="px-6 py-4">Nombre</th>
            <th class="px-6 py-4">Cédula</th>
            <th class="px-6 py-4">Carreña</th>
            <th class="px-6 py-4 text-center">Trayecto</th>
            <th class="px-6 py-4 text-right">Acciones</th>
          </tr>
        </thead>
        <tbody id="student-list" class="divide-y divide-gray-100">
          <!-- Datos cargados por JS -->
        </tbody>
      </table>
    </div>
    <div id="no-data" class="hidden p-12 text-center text-gray-500">
      No hay estudiantes registrados.
    </div>
  </div>
</div>
</div>
</div>

<!-- Toast de Notificaciones (Opcional pero recomendado para UX premium) -->
<div id="toast"
  class=
"fixed bottom-4 right-4 transform translate-y-20 opacity-0 transition-all duration-300 bg-gray-900 text-white px-6 py-3 rounded-lg shadow-2xl flex items-center space-x
-2"
>
  <span id="toast-msg">Mensaje informativo</span>
</div>

<script src="app.js"></script>
</body>
</html>

```

Creamos el archivo: public/app.js, que contiene la lógica del cliente. Realiza peticiones 'fetch' a la API del backend para poder listar, agregar, editar y eliminar estudiantes dinámicamente.

```

/**
 * Lógica principal del frontend.
 * Se encarga de la interacción con el DOM, Llamadas a La API y renderizado de datos.
 */
const API_URL = '/api/estudiantes';
const form = document.getElementById('student-form');
const studentList = document.getElementById('student-list');
const studentIdInput = document.getElementById('student-id');
const formTitle = document.getElementById('form-title');
const submitBtn = document.getElementById('submit-btn');
const cancelBtn = document.getElementById('cancel-btn');
const studentCountText = document.getElementById('student-count');
const noDataContainer = document.getElementById('no-data');

let isEditing = false;

// Cargar estudiantes al iniciar
document.addEventListener('DOMContentLoaded', fetchStudents);

async function fetchStudents() {
  try {
    const res = await fetch(API_URL);
    const data = await res.json();
    renderStudents(data);
  } catch (err) {
    showToast('Error al cargar datos', 'error');
  }
}

```



```

function renderStudents(students) {
  studentList.innerHTML = '';
  studentCountText.innerText = `${students.length} Estudiantes`;

  if (students.length === 0) {
    noDataContainer.classList.remove('hidden');
  } else {
    noDataContainer.classList.add('hidden');
    students.forEach(s => {
      const tr = document.createElement('tr');
      tr.className = 'hover:bg-gray-50 transition-colors';
      tr.innerHTML = `
        <td class="px-6 py-4 font-medium text-gray-900">${s.nombre}</td>
        <td class="px-6 py-4 text-gray-600">${s.cedula}</td>
        <td class="px-6 py-4 text-gray-600">${s.carrera}</td>
        <td class="px-6 py-4 text-center">
          <span class="bg-gray-100 text-gray-800 px-2 py-1 rounded text-xs font-bold">${s.trayecto_actual}</span>
        </td>
        <td class="px-6 py-4 text-right space-x-2">
          <button onClick="editStudent('${s._id}', '${s.nombre}', '${s.cedula}', '${s.carrera}', '${s.trayecto_actual}
)" class="text-indigo-600 hover:text-indigo-900 font-medium">Editar</button>
          <button onClick="deleteStudent('${s._id}')" class="text-red-600 hover:text-red-900 font-medium">Borrar</button>
        </td>
      `;
      studentList.appendChild(tr);
    });
  }
}

form.addEventListener('submit', async (e) => {
  e.preventDefault();

  const studentData = {
    nombre: document.getElementById('nombre').value,
    cedula: parseInt(document.getElementById('cedula').value),
    carrera: document.getElementById('carrera').value,
    trayecto_actual: parseInt(document.getElementById('trayecto').value)
  };

  try {
    let res;
    if (isEditing) {
      res = await fetch(`${API_URL}/${studentIdInput.value}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(studentData)
      });
    } else {
      res = await fetch(API_URL, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(studentData)
      });
    }

    if (res.ok) {
      showToast(isEditing ? '¡Actualizado con éxito!' : '¡Estudiante creado!');
      resetForm();
      fetchStudents();
    } else {
      const err = await res.json();
      showToast(err.message || 'Error en la operación', 'error');
    }
  } catch (err) {
    showToast('Error de conexión', 'error');
  }
});

function editStudent(id, nombre, cedula, carrera, trayecto) {
  isEditing = true;
  studentIdInput.value = id;
  document.getElementById('nombre').value = nombre;
  document.getElementById('cedula').value = cedula;
  document.getElementById('carrera').value = carrera;
  document.getElementById('trayecto').value = trayecto;

  formTitle.innerText = 'Editar Estudiante';
  submitBtn.innerText = 'Actualizar Datos';
  cancelBtn.classList.remove('hidden');

  // Scroll a tope para ver el formulario
  window.scrollTo({ top: 0, behavior: 'smooth' });
}

async function deleteStudent(id) {
  if (confirm('¿Estás seguro de eliminar a este estudiante?')) {
    try {
      const res = await fetch(`${API_URL}/${id}`, { method: 'DELETE' });
      if (res.ok) {
        showToast('Estudiante eliminado');
        fetchStudents();
      }
    } catch (err) {
      showToast('Error al eliminar', 'error');
    }
  }
}

```

```

function resetForm() {
  isEditing = false;
  form.reset();
  studentIdInput.value = '';
  formTitle.innerText = 'Agregar Estudiante';
  submitBtn.innerText = 'Guardar Estudiante';
  cancelBtn.classList.add('hidden');
}

cancelBtn.addEventListener('click', resetForm);

function showToast(msg, type = 'info') {
  const toast = document.getElementById('toast');
  const toastMsg = document.getElementById('toast-msg');

  toastMsg.innerText = msg;
  toast.classList.remove('translate-y-20', 'opacity-0');
  toast.classList.add('translate-y-0', 'opacity-100');

  setTimeout(() => {
    toast.classList.add('translate-y-20', 'opacity-0');
    toast.classList.remove('translate-y-0', 'opacity-100');
  }, 3000);
}

```

8. CÓMO EJECUTAR EL PROYECTO

Finalmente al momento de ejecutar la aplicación desarrollada, debemos:

- Modificar el archivo de configuración del proyecto package.json la sección de scripts que se crea por defecto y la sustituimos por:

```

"scripts": {
  "dev": "nodemon src/server.ts",
  "start": "ts-node src/server.ts",
  "build": "tsc"
},

```

- Asegurar que MongoDB este corriendo.
- Iniciar el servidor en modo desarrollo con el comando: **npm run dev**

```

PS C:\crud-node-ts-tailwind> npm run dev

> crud-node-ts-tailwind@1.0.0 dev
> nodemon src/server.ts

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: ts,json
[nodemon] starting `ts-node src/server.ts`
[dotenv@17.3.1] injecting env (2) from .env -- tip: 🔒 encrypt with Dotenvx: https://dotenvx.com
Conectado a MongoDB
Servidor corriendo en http://localhost:3000

```


- Luego abrimos en el navegador: <http://localhost:3000>

Sistema Escolar (Tarea SD3)
Gestión de Estudiantes (Node.js + TS + Tailwind + MongoDB)
UC Base de Datos II - Estudiantes: Ana Contreras, Diana Sierra y Darwin Colmenares

Agregar Estudiante

Nombre Completo

Cédula

Carrera

Trayecto Actual

Guardar Estudiante

Lista de Estudiantes 0 Estudiantes

NOMBRE	CÉDULA	CARRERA	TRAYECTO	ACCIONES
No hay estudiantes registrados.				

Cargamos varios registros para probar nuestra aplicación.

Sistema Escolar (Tarea SD3)
Gestión de Estudiantes (Node.js + TS + Tailwind + MongoDB)
UC Base de Datos II - Estudiantes: Ana Contreras, Diana Sierra y Darwin Colmenares

Agregar Estudiante

Nombre Completo

Cédula

Carrera

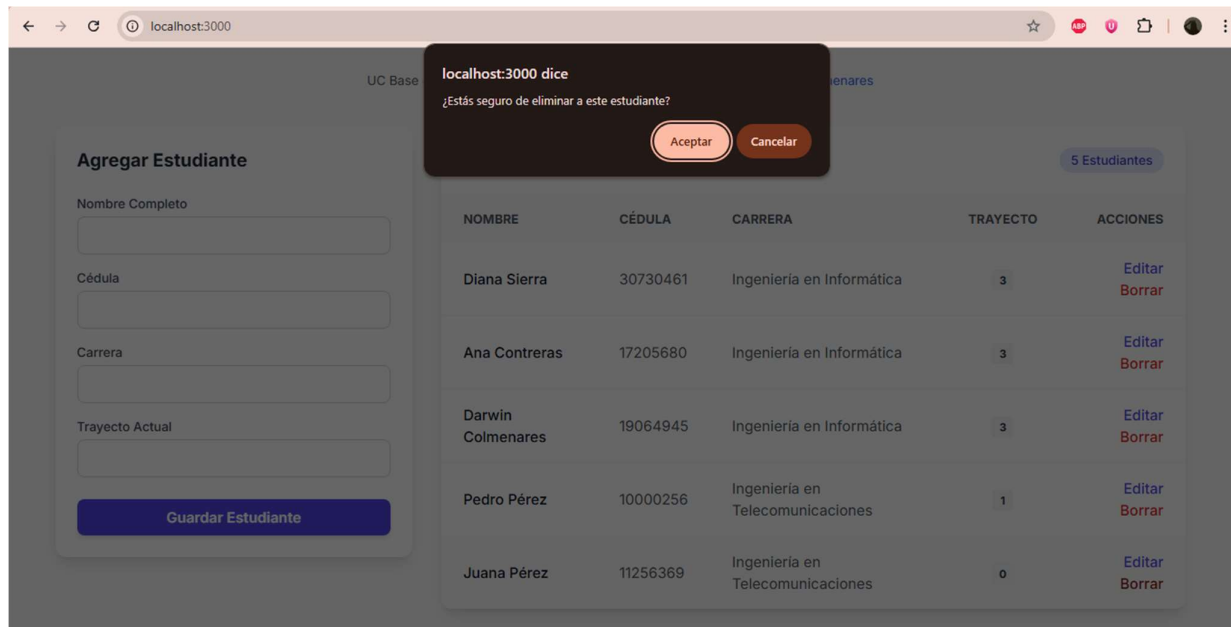
Trayecto Actual

Guardar Estudiante

Lista de Estudiantes 4 Estudiantes

NOMBRE	CÉDULA	CARRERA	TRAYECTO	ACCIONES
Diana Sierra	30730461	Ingeniería en Informática	3	Editar Borrar
Ana Contreras	17205680	Ingeniería en Informática	3	Editar Borrar
Darwin Colmenares	19064945	Ingeniería en Informática	3	Editar Borrar
Pedro Pérez	10000256	Ingeniería en Telecomunicaciones	1	Editar Borrar

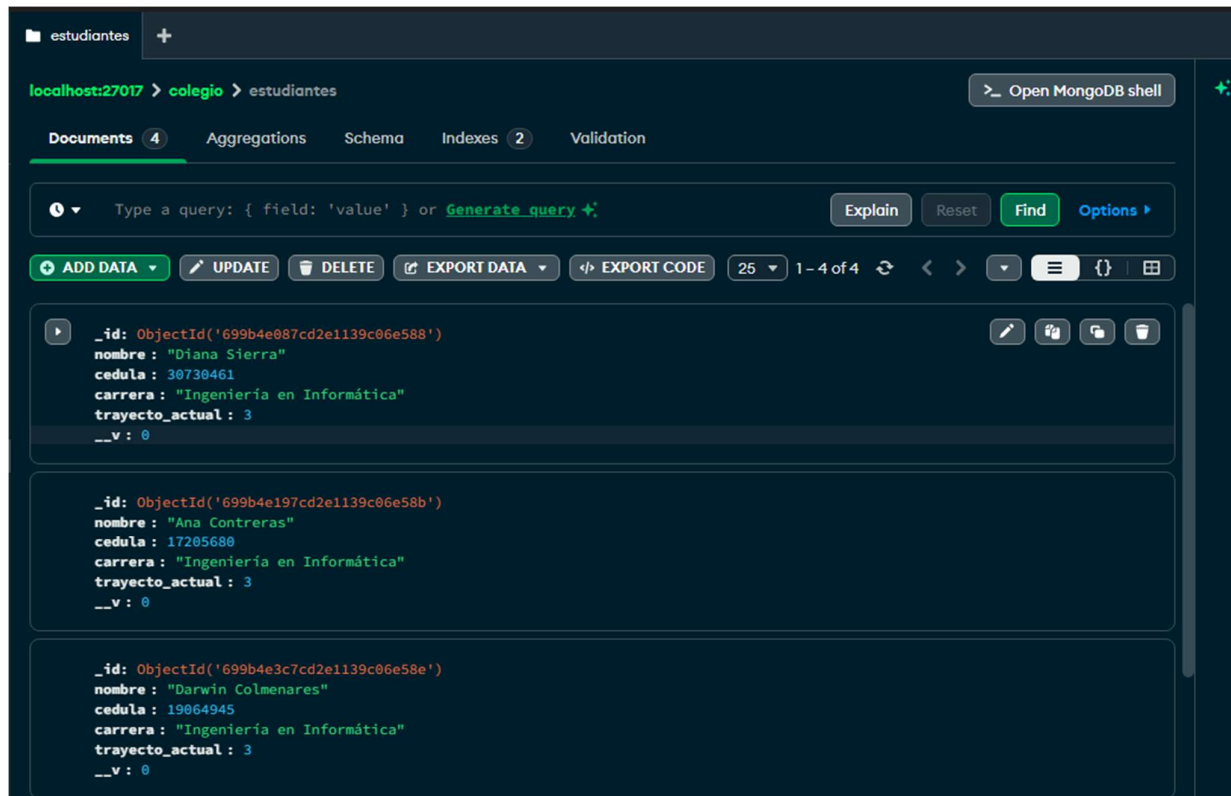
Borramos el quínte registro, Juana Pérez



Y se puede ver que se borró el registro:



Vista en la BD de la colección estudiantes con los registros insertados:



9. REPOSITORIO EN GITHUB

Para su revisión se creó un repositorio con los códigos de la aplicación creada.

<https://github.com/dianasierra1109/Evaluacion3BD.git>

CONCLUSIONES

Basados en los resultados reales del CRUD de estudiantes, concluimos:

- **Eficiencia en el Desarrollo:** La integración de Node.js y MongoDB permite un flujo de trabajo ágil, donde el formato JSON/BSON de los documentos coincide naturalmente con los objetos de JavaScript, reduciendo la complejidad en el mapeo de datos del sistema escolar.
- **Robustez con TypeScript:** El uso de TypeScript en el backend proporcionó una capa de seguridad adicional, permitiendo detectar errores en tiempo de compilación y asegurar que las interacciones con la base de datos sigan esquemas definidos, evitando datos inconsistentes en la colección de estudiantes.
- **Flexibilidad NoSQL:** Se comprobó que la naturaleza "schemaless" de MongoDB es ideal para aplicaciones educativas que puedan requerir la adición de campos dinámicos en el futuro (como nuevas calificaciones o datos de contacto) sin necesidad de migraciones complejas de base de datos.
- **Experiencia de Usuario:** La implementación de una interfaz con Tailwind CSS, conectada a la lógica del servidor, permitió validar que un sistema de gestión puede ser visualmente eficiente y técnicamente sólido bajo una arquitectura MVC bien estructurada.

REFERENCIAS BIBLIOGRÁFICAS

Casciaro, M., & Mammino, L. (2020). Node.js Design Patterns: Design and implement production-grade Node.js applications. Packt Publishing.

Microsoft. (2026). TypeScript Documentation: Stronger JavaScript at scale. Recuperado de: <https://www.typescriptlang.org/docs/>

MongoDB, Inc. (2026). Manual de referencia de MongoDB: Modelado de datos y CRUD. Recuperado de: <https://www.mongodb.com/docs/manual/>

OpenJS Foundation. (2026). Node.js v20.x Documentation. Recuperado de: <https://nodejs.org/docs/>